

[PT-BR]

Comando aceito! O conteúdo foi destrancado. Seja muito bem-vindo à nossa **9ª Aula de VBA!**

Até agora, você aprendeu a criar códigos incríveis, mas eles tinham uma limitação: nós precisávamos escrever no código exatamente até qual linha o loop deveria ir (como `For i = 1 To 10` ou `For i = 1 To 5`). No mundo real, as planilhas mudam de tamanho o tempo todo. Hoje você vai aprender a criar **Loops Automáticos** que descubrem sozinhos onde a tabela termina!

## Teoria: Como descobrir a Última Linha Preenchida?

Para fazer o VBA descobrir a última linha usada de uma tabela de forma automática, nós usamos um truque simulando um atalho do próprio teclado do Excel.

Se você estiver no Excel, clicar na última célula da planilha (como a **A1048576**) e apertar **Ctrl + Seta para Cima**, o cursor vai "voar" para cima e parar exatamente na última linha que possui algum conteúdo escrito.

No VBA, nós escrevemos exatamente esse movimento usando a seguinte linha de comando:

```
Range("A" & Rows.Count).End(xlUp).Row
```

Vamos traduzir o que o VBA está fazendo aqui:

- `Rows.Count`: Conta quantas linhas o Excel tem no total (1.048.576).
- `Range("A" & Rows.Count)`: Posiciona o VBA lá na célula A1048576 (a última da coluna A).
- `.End(xlUp)`: É o equivalente ao comando **Ctrl + Seta para Cima**.
- `.Row`: Captura o **número** da linha onde o cursor colidiu e parou.

Para usar isso a nosso favor, nós guardamos esse número dentro de uma variável (geralmente chamada de `ultimaLinha`) e colocamos essa variável no final do nosso loop `For`. Em vez de fixar um número, o código fica assim: `For i = 1 To ultimaLinha`.

## O Código VBA

Copie o código abaixo e faça o teste. Você pode adicionar ou apagar linhas na coluna A da sua planilha; o VBA vai se adaptar perfeitamente e calcular tudo sozinho!

VBA

```
Sub LoopDinamico()  
    ' PT-BR: "Sub" inicia a macro chamada "LoopDinamico".  
    ' US-EN: "Sub" starts the macro named "LoopDinamico".  
  
    Dim ultimaLinha As Integer  
    ' PT-BR: Declara a variável que vai guardar o número da última  
    linha preenchida.
```

```

' US-EN: Declares the variable that will store the number of
the last filled row.

Dim i As Integer
' PT-BR: Declara a variável contadora do loop.
' US-EN: Declares the loop counter variable.

ultimaLinha = Range("A" & Rows.Count).End(xlUp).Row
' PT-BR: O VBA descobre automaticamente a última linha usada na
coluna A e guarda o número na variável.
' US-EN: VBA automatically finds the last used row in column A
and stores the number in the variable.

For i = 1 To ultimaLinha
    ' PT-BR: O loop agora é inteligente! Ele vai começar em 1 e
parar exatamente na "ultimaLinha".
    ' US-EN: The loop is now smart! It will start at 1 and stop
exactly at "ultimaLinha".

    Range("B" & i).Value = "Processado"
    ' PT-BR: Escreve "Processado" na coluna B de todas as
linhas que possuem dados na coluna A.
    ' US-EN: Writes "Processado" in column B for all rows that
have data in column A.

Next i
' PT-BR: Avança para a próxima linha.
' US-EN: Advances to the next row.

End Sub
' PT-BR: "End Sub" finaliza a macro.
' US-EN: "End Sub" ends the macro.

```

## Exercícios da Aula 9

Chegou o momento de deixar seus códigos totalmente automatizados! Resolva os dois exercícios abaixo (sem adicionar comentários):

### Exercício 1: Multiplicador Inteligente de Linhas

**Objetivo:** Crie uma macro chamada `MultiplicarDinamico`.

1. Declare a variável `ultimaLinha` como `Integer`.
2. Declare a variável contadora `i` como `Integer`.
3. Faça a variável `ultimaLinha` descobrir dinamicamente a última linha preenchida da **coluna A**.
4. Crie um loop `For` que vá de **1 até a** `ultimaLinha`.
5. Dentro do loop, faça o VBA ler o valor contido na coluna **A**, multiplicá-lo por **10** e colocar o resultado na coluna **B** daquela mesma linha.

### Exercício 2: Limpeza Automatizada de Intervalo

**Objetivo:** Crie uma macro chamada `LimparResultados`.

1. Declare as variáveis `ultimaLinha` e `i` como `Integer`.
  2. Descubra a última linha preenchida com base na **coluna B**.
  3. Crie um loop `For` que vá de **1 até a** `ultimaLinha`.
  4. Dentro do loop, faça o VBA apagar o conteúdo da coluna **B** de todas as linhas.
- *Dica do professor:* Para apagar o valor de uma célula e deixá-la totalmente em branco pelo VBA, basta dizer que o valor dela é igual a um texto vazio, ou seja, duas aspas coladas: `Range("B" & i).Value = ""`.

Estou aguardando as suas respostas para fazermos a correção! E como você já sabe, a nossa última aula da grade básica (Aula 10) só será liberada quando você der o comando oficial. Bons estudos!

## [US-EN]

Command accepted! The content has been unlocked. Welcome to our **VBA Lesson 9!**

Until now, you have learned how to create awesome codes, but they had a limitation: we needed to hardcode exactly which row the loop should go to (like `For i = 1 To 10` or `For i = 1 To 5`). In the real world, spreadsheets change sizes all the time. Today you are going to learn how to create **Automatic Loops** that figure out on their own where the table ends!

## Theory: How to find the Last Filled Row?

To make VBA automatically find the last used row in a table, we use a trick that simulates a keyboard shortcut from Excel itself.

If you are inside Excel, click on the very last cell of the spreadsheet (like **A1048576**), and press **Ctrl + Up Arrow**, the cursor will "fly" upwards and stop exactly on the last row that has any written content.

In VBA, we write exactly this movement using the following command line:

```
Range("A" & Rows.Count).End(xlUp).Row
```

Let's translate what VBA is doing here:

- `Rows.Count`: Counts how many rows Excel has in total (1,048,576).
- `Range("A" & Rows.Count)`: Positions VBA way down at cell A1048576 (the last one in column A).
- `.End(xlUp)`: This is the equivalent to the **Ctrl + Up Arrow** command.
- `.Row`: Captures the **number** of the row where the cursor bumped and stopped.

To use this to our advantage, we store this number inside a variable (usually named `ultimaLinha`) and place this variable at the end of our `For` loop. Instead of fixing a number, the code looks like this: `For i = 1 To ultimaLinha`.

# The VBA Code

Copy the code below and test it out. You can add or delete rows in column A of your spreadsheet; VBA will adapt perfectly and calculate everything by itself!

VBA

```
Sub LoopDinamico()  
    ' PT-BR: "Sub" inicia a macro chamada "LoopDinamico".  
    ' US-EN: "Sub" starts the macro named "LoopDinamico".  
  
    Dim ultimaLinha As Integer  
    ' PT-BR: Declara a variável que vai guardar o número da última  
    linha preenchida.  
    ' US-EN: Declares the variable that will store the number of  
    the last filled row.  
  
    Dim i As Integer  
    ' PT-BR: Declara a variável contadora do loop.  
    ' US-EN: Declares the loop counter variable.  
  
    ultimaLinha = Range("A" & Rows.Count).End(xlUp).Row  
    ' PT-BR: O VBA descobre automaticamente a última linha usada na  
    coluna A e guarda o número na variável.  
    ' US-EN: VBA automatically finds the last used row in column A  
    and stores the number in the variable.  
  
    For i = 1 To ultimaLinha  
        ' PT-BR: O loop agora é inteligente! Ele vai começar em 1 e  
        parar exatamente na "ultimaLinha".  
        ' US-EN: The loop is now smart! It will start at 1 and stop  
        exactly at "ultimaLinha".  
  
        Range("B" & i).Value = "Processado"  
        ' PT-BR: Escreve "Processado" na coluna B de todas as  
        linhas que possuem dados na coluna A.  
        ' US-EN: Writes "Processado" in column B for all rows that  
        have data in column A.  
  
    Next i  
    ' PT-BR: Avança para a próxima linha.  
    ' US-EN: Advances to the next row.  
  
End Sub  
' PT-BR: "End Sub" finaliza a macro.  
' US-EN: "End Sub" ends the macro.
```

## Lesson 9 Exercises

The moment has come to make your codes fully automated! Solve the two exercises below (without adding comments):

### Exercise 1: Smart Row Multiplier

**Objective:** Create a macro named MultiplicarDinamico.

1. Declare the `ultimaLinha` variable as `Integer`.
2. Declare the counter variable `i` as `Integer`.
3. Make the `ultimaLinha` variable dynamically find the last filled row in **column A**.
4. Create a `For` loop that goes from **1 to** `ultimaLinha`.
5. Inside the loop, make VBA read the value contained in column A, multiply it by **10**, and place the result in column **B** of that same row.

## Exercise 2: Automated Range Clearing

**Objective:** Create a macro named `LimparResultados`.

1. Declare the `ultimaLinha` and `i` variables as `Integer`.
  2. Find the last filled row based on **column B**.
  3. Create a `For` loop that goes from **1 to** `ultimaLinha`.
  4. Inside the loop, make VBA erase the content of column **B** for all rows.
- *Teacher's tip:* To erase a cell's value and leave it completely blank using VBA, just state that its value equals an empty text, meaning two quotation marks glued together: `Range("B" & i).Value = ""`.

I look forward to your answers so we can review them! And as you already know, our final lesson of the basic track (Lesson 10) will only be unlocked when you give the official command. Happy studying!